

Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа №2

По дисциплине «**Распределённые системы хранения данных**»

Выполнил:

Студент группы Р3306,
Михайлов Дмитрий Андреевич

Преподаватель:

Максимов Андрей Николаевич



Санкт-Петербург

2026 год

Содержание

Задание	3
Подготовка окружения	4
Этап 1. Резервное копирование.....	5
1.1. Проверка основного узла.....	5
1.2. Подготовка каталога на резервном узле	5
1.3. Настройка SSH-доступа для автоматической передачи архивов	5
1.4. Сценарий резервного копирования	6
1.5. Проверка сценария резервного копирования	7
1.6. Настройка периодического запуска через cron	8
1.7. Расчет объема резервных копий за месяц.....	9
1.8. Анализ результатов этапа.....	9
Этап 2. Потеря основного узла	10
2.1. Подготовка резервного узла и выбор резервной копии.....	10
2.2. Инициализация восстановленного кластера	11
2.3. Конфигурация и запуск сервера на резервном узле	12
2.4. Восстановление из SQL Dump-архива	13
2.5. Проверка доступности данных	14
2.6. Анализ результатов этапа.....	16
Этап 3. Повреждение файлов БД	17
3.1. Проверка исходного состояния основного узла	17
3.2. Симуляция сбоя	19
3.3. Подготовка восстановления в новый PGDATA	20
3.4. Инициализация и конфигурация нового кластера.....	21
3.5. Восстановление данных из резервной копии	23
3.6. Проверка результата восстановления	24
3.7. Анализ результатов этапа.....	26
Этап 4. Логическое повреждение данных.....	27
4.1. Подготовка основного узла и включение архивирования WAL.....	27
4.2. Создание физической базовой копии	29
4.3. Добавление новых строк и фиксация точки восстановления	29
4.4. Симуляция логической ошибки	31
4.5. Восстановление на момент времени с использованием архивных WAL.....	32
4.6. Проверка результата восстановления	33
4.7. Анализ результатов этапа.....	35

Задание

Цель работы - настроить процедуру периодического резервного копирования базы данных, сконфигурированной в ходе выполнения лабораторной работы №2, а также разработать и отладить сценарии восстановления в случае сбоя.

Узел из предыдущей лабораторной работы используется в качестве основного. Новый узел используется в качестве резервного. Учётные данные для подключения к новому узлу выдаёт преподаватель. В сценариях восстановления необходимо использовать копию данных, полученную на первом этапе данной лабораторной работы.

Требования к отчёту

Отчет должен быть самостоятельным документом (без ссылок на внешние ресурсы), содержать всю последовательность команд и исходный код скриптов по каждому пункту задания. Для демонстрации результатов приводить команду вместе с выводом (самой наглядной частью вывода, при необходимости).

Этап 1. Резервное копирование

- Настроить резервное копирование с основного узла на резервный следующим образом:

Периодические полные копии с помощью SQL Dump.

По расписанию (cron) раз в сутки, методом SQL Dump с сжатием. Созданные архивы должны сразу перемещаться на резервный хост, они не должны храниться на основной системе. Срок хранения архивов на резервной системе - 4 недели. По истечении срока хранения, старые архивы должны автоматически уничтожаться.

- Подсчитать, каков будет объем резервных копий спустя месяц работы системы, исходя из следующих условий:
 - Средний объем новых данных в БД за сутки: 650МБ.
 - Средний объем измененных данных за сутки: 750МБ.
- Проанализировать результаты.

Этап 2. Потеря основного узла

Этот сценарий подразумевает полную недоступность основного узла. Необходимо восстановить работу СУБД на РЕЗЕРВНОМ узле, продемонстрировать успешный запуск СУБД и доступность данных.

Этап 3. Повреждение файлов БД

Этот сценарий подразумевает потерю данных (например, в результате сбоя диска или файловой системы) при сохранении доступности основного узла. Необходимо выполнить полное восстановление данных из резервной копии и перезапустить СУБД на ОСНОВНОМ узле.

Ход работы:

- Симулировать сбой:
 - удалить с диска директорию любого табличного пространства со всем содержимым.
- Проверить работу СУБД, доступность данных, перезапустить СУБД, проанализировать результаты.

- Выполнить восстановление данных из резервной копии, учитывая следующее условие:
 - исходное расположение директории PGDATA недоступно - разместить данные в другой директории и скорректировать конфигурацию.
- Запустить СУБД, проверить работу и доступность данных, проанализировать результаты.

Этап 4. Логическое повреждение данных

Этот сценарий подразумевает частичную потерю данных (в результате нежелательной или ошибочной операции) при сохранении доступности основного узла. Необходимо выполнить восстановление данных на ОСНОВНОМ узле следующим способом:

- Восстановление с использованием архивных WAL файлов. (СУБД должна работать в режиме архивирования WAL, потребуется задать параметры восстановления).

Ход работы:

- В каждую таблицу базы добавить 2-3 новые строки, зафиксировать результат.
- Зафиксировать время и симулировать ошибку:
 - удалить любые две таблицы (DROP TABLE)
- Продемонстрировать результат.
- Выполнить восстановление данных указанным способом.
- Продемонстрировать и проанализировать результат.

Подготовка окружения

```
mysticslippers@LAPTOP-GD639C6U:~$ ssh -J s368530@helios.cs.ifmo.ru:2222
postgres3@pg111

[postgres3@pg111 ~]$ export PGDATA="$HOME/ymy46"      #Директория кластера
[postgres3@pg111 ~]$ export PGPORT="9530"           #Номер порта
[postgres3@pg111 ~]$ export PGDATABASE="fastorangecity"
```

Этап 1. Резервное копирование

Основной узел: postgres3@pg111. Резервный узел: postgres4@pg121. Кластер PostgreSQL на основном узле был создан в предыдущей лабораторной работе в каталоге \$HOME/ymy46 и работает на порту 9530. Целевая база данных: fastorangecity.

1.1. Проверка основного узла

Сначала на основном узле были заданы переменные окружения для работы с экземпляром PostgreSQL:

```
[postgres3@pg111 ~]$ export PGDATA="$HOME/ymy46"
[postgres3@pg111 ~]$ export PGPORT="9530"
[postgres3@pg111 ~]$ export PGDATABASE="fastorangecity"
```

После этого был выполнен запуск сервера PostgreSQL:

```
[postgres3@pg111 ~]$ pg_ctl -D "$PGDATA" start
ожидание запуска сервера...2026-04-26 10:27:23.387 MSK [61090] СООБЩЕНИЕ: передача
вывода в протокол процессу сбора протоколов
2026-04-26 10:27:23.387 MSK [61090] ПОДСКАЗКА: В дальнейшем протоколы будут
выводиться в каталог "log".
    Готово
сервер запущен
```

Доступность базы данных была проверена с помощью psql:

```
[postgres3@pg111 ~]$ psql -p "$PGPORT" -d fastorangecity -c '\conninfo'
Вы подключены к базе данных "fastorangecity" как пользователь "postgres3" через сокет
в "/tmp", порт "9530".
```

Таким образом, основной экземпляр PostgreSQL был запущен и база данных fastorangecity доступна для подключения.

1.2. Подготовка каталога на резервном узле

На резервном узле был создан отдельный каталог для хранения сжатых SQL Dump-архивов:

```
mysticslippers@LAPTOP-GD639C6U:~$ ssh -J s368530@helios.cs.ifmo.ru:2222
postgres4@pg121

[postgres4@pg121 ~]$ mkdir -p "$HOME/pg_backups/sql_dump"
[postgres4@pg121 ~]$ chmod 700 "$HOME/pg_backups" "$HOME/pg_backups/sql_dump"
[postgres4@pg121 ~]$ ls -ld "$HOME/pg_backups/sql_dump"
drwx----- 2 postgres4 postgres 2 26 apr. 10:31
/var/db/postgres4/pg_backups/sql_dump
```

1.3. Настройка SSH-доступа для автоматической передачи архивов

Так как резервное копирование должно выполняться автоматически по расписанию, был настроен SSH-доступ по ключу с основного узла на резервный. Это необходимо, так как cron не может интерактивно вводить пароль.

На основном узле был создан ключ для передачи резервных копий:

```
[postgres3@pg111 ~]$ mkdir -p ~/.ssh
[postgres3@pg111 ~]$ chmod 700 ~/.ssh
[postgres3@pg111 ~]$ ssh-keygen -t ed25519 -f ~/.ssh/backup_pg121_key -N ""
Generating public/private ed25519 key pair.
Your identification has been saved in /var/db/postgres3/.ssh/backup_pg121_key
Your public key has been saved in /var/db/postgres3/.ssh/backup_pg121_key.pub
```

```
The key fingerprint is:  
SHA256:uPxDf1HK4BAWi6H3tLZgWalfcWl12doU9d+R8qEKiOI postgres3@pg111.cs.ifmo.ru
```

Публичный ключ был добавлен в файл ~/.ssh/authorized_keys на резервном узле pg121 и на промежуточном узле helios. Это позволило выполнять подключение по цепочке pg111 -> helios -> pg121 без ввода пароля.

Для удобства был создан SSH-конфигурационный файл на основном узле:

```
[postgres3@pg111 ~]$ cat > ~/.ssh/config <<'EOF'  
Host helios_jump  
  HostName helios.cs.ifmo.ru  
  Port 2222  
  User s368530  
  IdentityFile ~/.ssh/backup_pg121_key  
  IdentitiesOnly yes  
  
Host backup_pg121  
  HostName pg121  
  User postgres4  
  ProxyJump helios_jump  
  IdentityFile ~/.ssh/backup_pg121_key  
  IdentitiesOnly yes  
  ServerAliveInterval 30  
  ServerAliveCountMax 3  
EOF  
  
[postgres3@pg111 ~]$ chmod 600 ~/.ssh/config
```

Была выполнена проверка подключения к промежуточному узлу helios:

```
[postgres3@pg111 ~]$ ssh helios_jump 'hostname; whoami'  
helios.cs.ifmo.ru  
s368530
```

После этого была проверена полная цепочка подключения до резервного узла:

```
[postgres3@pg111 ~]$ ssh backup_pg121 'hostname; pwd; ls -ld ~/pg_backups/sql_dump'  
pg121.cs.ifmo.ru  
/var/db/postgres4  
drwx----- 2 postgres4 postgres 2 26 апр. 10:31  
/var/db/postgres4/pg_backups/sql_dump
```

Команда выполнилась без запроса пароля, следовательно, автоматическая передача архивов с основного узла на резервный возможна из cron.

1.4. Сценарий резервного копирования

На основном узле был создан каталог для сценариев и журналов выполнения:

```
[postgres3@pg111 ~]$ mkdir -p "$HOME/scripts" "$HOME/backup_logs"
```

Далее был создан сценарий backup_sql_dump.sh:

```
[postgres3@pg111 ~]$ cat > "$HOME/scripts/backup_sql_dump.sh" <<'EOF'  
#!/bin/sh  
  
set -eu  
  
export PGDATA="$HOME/ymy46"  
export PGPORT="9530"  
  
BACKUP_HOST="backup_pg121"  
REMOTE_DIR="/var/db/postgres4/pg_backups/sql_dump"
```

```

DATE_TAG="$(date '+%Y-%m-%d_%H-%M-%S')"
BACKUP_NAME="pg111_full_cluster_${DATE_TAG}.sql.gz"
LOG_FILE="$HOME/backup_logs/backup_sql_dump.log"

echo "[$(date '+%Y-%m-%d %H:%M:%S')] backup started: ${BACKUP_NAME}" >> "$LOG_FILE"

pg_ctl -D "$PGDATA" status >> "$LOG_FILE" 2>&1

ssh "$BACKUP_HOST" "mkdir -p '$REMOTE_DIR' && chmod 700 '$REMOTE_DIR'"

pg_dumpall -p "$PGPORT" \
| gzip -9 \
| ssh "$BACKUP_HOST" "cat > '$REMOTE_DIR/${BACKUP_NAME}.tmp' && mv '$REMOTE_DIR/${BACKUP_NAME}.tmp' '$REMOTE_DIR/${BACKUP_NAME}'"

ssh "$BACKUP_HOST" "find '$REMOTE_DIR' -type f -name 'pg111_full_cluster_*.sql.gz' -mtime +28 -delete"

ssh "$BACKUP_HOST" "ls -lh '$REMOTE_DIR/${BACKUP_NAME}'" >> "$LOG_FILE" 2>&1

echo "[$(date '+%Y-%m-%d %H:%M:%S')] backup finished: ${BACKUP_NAME}" >> "$LOG_FILE"
EOF

[postgres3@pg111 ~]$ chmod +x "$HOME/scripts/backup_sql_dump.sh"

```

Сценарий работает потоковым способом: вывод `pg_dumpall` передается в `gzip`, после чего сразу записывается на резервный узел через SSH. Поэтому архив резервной копии не создается и не хранится на основном узле. Для защиты от частично записанных файлов сначала создается временный файл с расширением `.tmp`, а затем он переименовывается в итоговый архив.

Удаление архивов старше 28 дней выполняется командой `find` на резервном узле:

```
find '$REMOTE_DIR' -type f -name 'pg111_full_cluster_*.sql.gz' -mtime +28 -delete
```

1.5. Проверка сценария резервного копирования

После создания сценария был выполнен тестовый запуск:

```
[postgres3@pg111 ~]$ "$HOME/scripts/backup_sql_dump.sh"
```

Журнал выполнения показал успешное создание и передачу архива на резервный узел:

```

[postgres3@pg111 ~]$ tail -n 20 "$HOME/backup_logs/backup_sql_dump.log"
[2026-04-26 11:22:51] backup started: pg111_full_cluster_2026-04-26_11-22-51.sql.gz
pg_ctl: сервер работает (PID: 61090)
/usr/local/bin/postgres "-D" "/var/db/postgres3/ymy46"
-rw-r--r--  1 postgres4 postgres  2,4K 26 апр.  11:22
/var/db/postgres4/pg_backups/sql_dump/pg111_full_cluster_2026-04-26_11-22-51.sql.gz
[2026-04-26 11:22:55] backup finished: pg111_full_cluster_2026-04-26_11-22-51.sql.gz
[2026-04-26 11:26:12] backup started: pg111_full_cluster_2026-04-26_11-26-12.sql.gz
pg_ctl: сервер работает (PID: 61090)
/usr/local/bin/postgres "-D" "/var/db/postgres3/ymy46"
-rw-r--r--  1 postgres4 postgres  2,4K 26 апр.  11:26
/var/db/postgres4/pg_backups/sql_dump/pg111_full_cluster_2026-04-26_11-26-12.sql.gz
[2026-04-26 11:26:15] backup finished: pg111_full_cluster_2026-04-26_11-26-12.sql.gz

```

Наличие архивов на резервном узле было проверено командой:

```

[postgres3@pg111 ~]$ ssh backup_pg121 'ls -lh /var/db/postgres4/pg_backups/sql_dump'
total 9
-rw-r--r--  1 postgres4 postgres  2,4K 26 апр.  11:22 pg111_full_cluster_2026-04-26_11-22-51.sql.gz

```

```
-rw-r--r-- 1 postgres4 postgres 2,4K 26 апр. 11:26 pg111_full_cluster_2026-04-26_11-26-12.sql.gz
```

Проверка отсутствия архивов на основном узле:

```
[postgres3@pg111 ~]$ find "$HOME" -maxdepth 3 -name 'pg111_full_cluster_*.sql.gz'
```

Команда не вывела результатов. Следовательно, архивы не сохраняются на основной системе и сразу передаются на резервный узел.

Также была проверена целостность последнего gzip-архива:

```
[postgres3@pg111 ~]$ ssh backup_pg121 'LATEST=$(ls -t /var/db/postgres4/pg_backups/sql_dump/pg111_full_cluster_*.sql.gz | head -n 1); echo "$LATEST"; gzip -t "$LATEST" && echo OK' /var/db/postgres4/pg_backups/sql_dump/pg111_full_cluster_2026-04-26_11-26-12.sql.gz OK
```

Проверка содержимого архива показала, что внутри находится SQL Dump всего кластера PostgreSQL:

```
[postgres3@pg111 ~]$ ssh backup_pg121 'LATEST=$(ls -t /var/db/postgres4/pg_backups/sql_dump/pg111_full_cluster_*.sql.gz | head -n 1); gzip -dc "$LATEST" | head -n 20'
--
-- PostgreSQL database cluster dump
--

SET default_transaction_read_only = off;

SET client_encoding = 'WIN1251';
SET standard_conforming_strings = on;

--
-- Roles
--

CREATE ROLE default_user;
ALTER ROLE default_user WITH NOSUPERUSER INHERIT NOCREATEROLE NOCREATEDB LOGIN
NOREPLICATION NOBYPASSRLS PASSWORD 'SCRAM-SHA-256$...';
CREATE ROLE postgres3;
ALTER ROLE postgres3 WITH SUPERUSER INHERIT CREATEROLE CREATEDB LOGIN REPLICATION
BYPASSRLS;
```

В начале дампа присутствует строка SET client_encoding = 'WIN1251', что соответствует кодировке кластера, заданной при выполнении предыдущей лабораторной работы. Также в дампе присутствует секция Roles, что подтверждает использование pg_dumpall и сохранение глобальных объектов кластера.

1.6. Настройка периодического запуска через cron

Для ежедневного автоматического запуска сценария резервного копирования было создано задание cron. Чтобы избежать ошибок при редактировании crontab, запись была добавлена одной командой:

```
[postgres3@pg111 ~]$ ( crontab -l 2>/dev/null; echo '0 3 * * * /var/db/postgres3/scripts/backup_sql_dump.sh >> /var/db/postgres3/backup_logs/cron_backup.log 2>&1' ) | crontab -
```

Проверка установленного расписания:

```
[postgres3@pg111 ~]$ crontab -l
```



```
0 3 * * * /var/db/postgres3/scripts/backup_sql_dump.sh >>  
/var/db/postgres3/backup_logs/cron_backup.log 2>&1
```

Задание cron выполняет сценарий backup_sql_dump.sh ежедневно в 03:00. Стандартный вывод и сообщения об ошибках перенаправляются в файл /var/db/postgres3/backup_logs/cron_backup.log.

1.7. Расчет объема резервных копий за месяц

По условию задания средний объем новых данных в базе данных за сутки составляет 650 МБ, а средний объем измененных данных за сутки - 750 МБ. При этом используется полное логическое резервное копирование SQL Dump. Это означает, что каждый ежедневный архив содержит полное состояние кластера на момент копирования, а не только изменения за сутки.

Пусть S_0 - начальный размер логического дампа. Тогда размер полного дампа в n -й день можно оценить как:

$$S(n) = S_0 + 650 * n \text{ МБ}$$

Так как срок хранения архивов на резервном узле составляет 4 недели, одновременно будут храниться последние 28 ежедневных копий. Через месяц работы системы на резервном узле останутся копии за дни с 3-го по 30-й. Тогда суммарный объем без учета сжатия равен:

$$V = \sum(S_0 + 650 * n), \text{ где } n = 3..30$$

$$V = 28 * S_0 + 650 * (3 + 4 + \dots + 30)$$

$$3 + 4 + \dots + 30 = 462$$

$$V = 28 * S_0 + 650 * 462$$

$$V = 28 * S_0 + 300300 \text{ МБ}$$

Если не учитывать начальный размер базы, прирост суммарного объема хранимых полных резервных копий за месяц составит:

$$300300 \text{ МБ} \approx 293,26 \text{ ГБ}$$

Реальный объем на резервном узле будет меньше из-за gzip-сжатия. Его можно оценить по фактическому коэффициенту сжатия первого архива. В тестовом запуске размер архива составил 2,4 КБ, так как база на момент проверки была небольшой.

Средний объем измененных данных за сутки 750 МБ не прибавляется как отдельный ежедневный инкремент, потому что в данном этапе используется не инкрементальное резервное копирование, а полный SQL Dump. Измененные строки входят в очередную полную копию уже в новом состоянии. Если бы использовалось архивирование WAL или инкрементальные резервные копии, измененные данные существенно влияли бы на объем хранимых журналов или инкрементов.

1.8. Анализ результатов этапа

В результате выполнения этапа была настроена процедура ежедневного полного логического резервного копирования кластера PostgreSQL с основного узла pg111 на резервный узел pg121. Резервная копия создается с помощью pg_dumpall, сжимается через gzip и сразу передается на резервный узел по SSH. На основной системе архив не сохраняется, что соответствует требованиям задания.

На резервном узле настроено хранение архивов в каталоге `/var/db/postgres4/pg_backups/sql_dump`. Устаревшие архивы старше 28 дней автоматически удаляются при каждом запуске сценария. Тестовый запуск подтвердил создание архива, его наличие на резервном узле, отсутствие архива на основном узле, целостность gzip-файла и наличие внутри SQL Dump-копии кластера.

Главный недостаток выбранного способа - рост суммарного объема хранения при ежедневных полных копиях. Даже без учета начального размера базы за месяц работы при ежедневном приросте 650 МБ новых данных потребуется около 293,26 ГБ до сжатия. Однако такой подход прост в реализации и удобен для восстановления, так как каждая резервная копия является самодостаточным SQL-скриптом восстановления состояния кластера на момент создания копии.

Этап 2. Потеря основного узла

На данном этапе был рассмотрен сценарий полной недоступности основного узла `pg111`. Восстановление работы СУБД выполнялось на резервном узле `pg121` с использованием SQL Dump-архива, полученного на первом этапе лабораторной работы.

Так как резервная копия была создана с помощью `pg_dumpall`, архив содержал SQL-скрипт восстановления всего кластера, включая роли, базы данных, табличные пространства и объекты пользовательской базы `fastorangecity`. Перед восстановлением на резервном узле был подготовлен новый каталог данных PostgreSQL, отдельный каталог WAL и каталог для пользовательского табличного пространства `idxspace`.

2.1. Подготовка резервного узла и выбор резервной копии

На резервном узле были заданы переменные окружения для нового восстановленного экземпляра PostgreSQL:

```
[postgres4@pg121 ~]$ export PGDATA="$HOME/ymy46_restored"
[postgres4@pg121 ~]$ export PGPORT="9530"
[postgres4@pg121 ~]$ export PGDATABASE="fastorangecity"
[postgres4@pg121 ~]$ BACKUP_DIR="$HOME/pg_backups/sql_dump"
[postgres4@pg121 ~]$ TABLESPACE_DIR="$HOME/mgg73"
```

Был просмотрен каталог с резервными копиями, созданными на первом этапе:

```
[postgres4@pg121 ~]$ ls -lh "$BACKUP_DIR"
total 14
-rw-r--r--  1 postgres4 postgres  2,4K 26 апр.  11:22 pg111_full_cluster_2026-04-26_11-22-51.sql.gz
-rw-r--r--  1 postgres4 postgres  2,4K 26 апр.  11:26 pg111_full_cluster_2026-04-26_11-26-12.sql.gz
-rw-r--r--  1 postgres4 postgres  2,4K 27 апр.   03:00 pg111_full_cluster_2026-04-27_03-00-00.sql.gz
```

Для восстановления был выбран последний по времени архив:

```
[postgres4@pg121 ~]$ LATEST_BACKUP="$(ls -t
"$BACKUP_DIR"/pg111_full_cluster_*.sql.gz | head -n 1)"
[postgres4@pg121 ~]$ echo "$LATEST_BACKUP"
```

```
/var/db/postgres4/pg_backups/sql_dump/pg111_full_cluster_2026-04-27_03-00-00.sql.gz
```

Целостность архива была проверена с помощью gzip:

```
[postgres4@pg121 ~]$ gzip -t "$LATEST_BACKUP" && echo "backup archive is valid"
backup archive is valid
```

Также было проверено начало SQL Dump-архива:

```
[postgres4@pg121 ~]$ gzip -dc "$LATEST_BACKUP" | head -n 20
--
-- PostgreSQL database cluster dump
--

SET default_transaction_read_only = off;

SET client_encoding = 'WIN1251';
SET standard_conforming_strings = on;

--
-- Roles
--

CREATE ROLE default_user;
ALTER ROLE default_user WITH NOSUPERUSER INHERIT NOCREATEROLE NOCREATEDB LOGIN
NOREPLICATION NOBYPASSRLS PASSWORD 'SCRAM-SHA-256$...';
CREATE ROLE postgres3;
ALTER ROLE postgres3 WITH SUPERUSER INHERIT CREATEROLE CREATEDB LOGIN REPLICATION
BYPASSRLS;
```

Из вывода видно, что архив является дампом кластера PostgreSQL, использует кодировку WIN1251 и содержит восстановление ролей default_user и postgres3.

Далее были подготовлены каталоги для нового кластера и пользовательского табличного пространства:

```
[postgres4@pg121 ~]$ mkdir -p "$PGDATA"
[postgres4@pg121 ~]$ mkdir -p "$TABLESPACE_DIR"
[postgres4@pg121 ~]$ chmod 700 "$PGDATA" "$TABLESPACE_DIR"
[postgres4@pg121 ~]$ ls -ld "$PGDATA" "$TABLESPACE_DIR"
drwx----- 2 postgres4 postgres 2 1 мая 12:55 /var/db/postgres4/mgg73
drwx----- 2 postgres4 postgres 2 1 мая 12:55 /var/db/postgres4/ymy46_restored
```

2.2. Инициализация восстановленного кластера

Для соответствия конфигурации основного узла был подготовлен отдельный каталог WAL. В предыдущей лабораторной работе основной кластер использовал отдельную директорию WAL, поэтому при восстановлении на резервном узле была сохранена аналогичная структура каталогов.

```
[postgres4@pg121 ~]$ export PGWAL="$HOME/xlg69_restored"
[postgres4@pg121 ~]$ mkdir -p "$PGWAL"
[postgres4@pg121 ~]$ chmod 700 "$PGWAL"
```

Новый кластер был инициализирован с той же локалью и кодировкой, что и исходный кластер:

```
[postgres4@pg121 ~]$ rmdir "$PGDATA" 2>/dev/null || true

[postgres4@pg121 ~]$ initdb -D "$PGDATA" -X "$PGWAL" --locale=ru_RU.CP1251 -
-encoding=WIN1251
Файлы, относящиеся к этой СУБД, будут принадлежать пользователю "postgres4".
От его имени также будет запускаться процесс сервера.

Кластер баз данных будет инициализирован с локалью "ru_RU.CP1251".
Выбрана конфигурация текстового поиска по умолчанию "russian".

создание каталога /var/db/postgres4/ymy46_restored... ок
исправление прав для существующего каталога /var/db/postgres4/xlg69_restored... ок
создание подкаталогов... ок
создание конфигурационных файлов... ок
сохранение данных на диске... ок

Готово. Теперь вы можете запустить сервер баз данных:

pg_ctl -D /var/db/postgres4/ymy46_restored -l файл_журнала start
```

Таким образом, на резервном узле был создан новый пустой кластер PostgreSQL с локалью ru_RU.CP1251 и кодировкой WIN1251, совместимый с дампом основного кластера.

2.3. Конфигурация и запуск сервера на резервном узле

После инициализации в файл postgresql.conf были добавлены параметры, аналогичные параметрам основного узла из предыдущей лабораторной работы:

```
[postgres4@pg121 ~]$ cat >> "$PGDATA/postgresql.conf" <<'EOF'

# Настройки восстановленного экземпляра для этапа 2
listen_addresses = 'localhost'
port = 9530

max_connections = 30
shared_buffers = 2GB
temp_buffers = 32MB
work_mem = 64MB
checkpoint_timeout = 15min
effective_cache_size = 18GB
fsync = on
commit_delay = 10000

logging_collector = on
log_destination = 'stderr'
log_directory = 'log'
log_filename = 'postgresql-%Y-%m-%d_%H%M%S.log'
log_min_messages = notice
log_connections = on
log_checkpoints = on
EOF
```

Файл pg_hba.conf был настроен для локальных подключений. Подключения через Unix-domain socket выполняются в режиме peer, TCP/IP-подключения разрешены только с localhost, остальные сетевые подключения явно запрещены:

```
[postgres4@pg121 ~]$ cat > "$PGDATA/pg_hba.conf" <<'EOF'
# TYPE DATABASE USER ADDRESS METHOD
```

```
# Unix-domain socket
local all all peer

# TCP/IP only from localhost
host all all 127.0.0.1/32 trust
host all all ::1/128 trust

# Reject everything else
host all all 0.0.0.0/0 reject
host all all ::0/0 reject
EOF
```

После настройки сервер PostgreSQL был запущен на резервном узле:

```
[postgres4@pg121 ~]$ pg_ctl -D "$PGDATA" start
ожидание запуска сервера...2026-05-01 12:58:32.389 MSK [39786] СООБЩЕНИЕ:
передача вывода в протокол процессу сбора протоколов
2026-05-01 12:58:32.389 MSK [39786] ПОДСКАЗКА: В дальнейшем протоколы будут
выводиться в каталог "log".
    готово
сервер запущен

[postgres4@pg121 ~]$ pg_ctl -D "$PGDATA" status
pg_ctl: сервер работает (PID: 39786)
/usr/local/bin/postgres "-D" "/var/db/postgres4/ymy46_restored"

[postgres4@pg121 ~]$ psql -p "$PGPORT" -d postgres -c '\conninfo'
Вы подключены к базе данных "postgres" как пользователь "postgres4" через сокет в
"/tmp", порт "9530".
```

2.4. Восстановление из SQL Dump-архива

Перед восстановлением было проверено, что в дампе присутствует старый путь пользовательского табличного пространства основного узла:

```
[postgres4@pg121 ~]$ gzip -dc "$LATEST_BACKUP" | grep -n "CREATE
TABLESPACE\|/var/db/postgres3/mgg73" | head -n 20
32:CREATE TABLESPACE idxspace OWNER postgres3 LOCATION '/var/db/postgres3/mgg73';
```

Так как восстановление выполнялось на другом узле и под другим системным пользователем, путь табличного пространства был заменён с /var/db/postgres3/mgg73 на /var/db/postgres4/mgg73. Восстановление было выполнено потоковой командой:

```
[postgres4@pg121 ~]$ gzip -dc "$LATEST_BACKUP" | sed
"s|/var/db/postgres3/mgg73|/var/db/postgres4/mgg73|g" | psql -p "$PGPORT" -d
postgres -v ON_ERROR_STOP=1
SET
SET
SET
CREATE ROLE
ALTER ROLE
CREATE ROLE
ALTER ROLE
CREATE TABLESPACE
GRANT
Вы подключены к базе данных "template1" как пользователь "postgres4".
...
CREATE DATABASE
ALTER DATABASE
```

```

Вы подключены к базе данных "fastorangecity" как пользователь "postgres4".
...
CREATE TABLE
ALTER TABLE
CREATE SEQUENCE
ALTER SEQUENCE
COPY 3
COPY 3
COPY 3
COPY 4
CREATE INDEX
CREATE INDEX
CREATE INDEX
CREATE INDEX
Вы подключены к базе данных "postgres" как пользователь "postgres4".

```

2.5. Проверка доступности данных

После восстановления был проверен список баз данных:

```

[postgres4@pg121 ~]$ psql -p "$PGPORT" -d postgres -c '\l'

```

Имя	Владелец	Кодировка	Провайдер локали	Список баз данных LC_COLLATE
fastorangecity	default_user	WIN1251	libc	ru_RU.CP1251
postgres	postgres4	WIN1251	libc	ru_RU.CP1251
template0	postgres4	WIN1251	libc	ru_RU.CP1251
template1	postgres4	WIN1251	libc	ru_RU.CP1251

(4 строки)

Также был проверен список табличных пространств:

```

[postgres4@pg121 ~]$ psql -p "$PGPORT" -d postgres -c '\db'

```

Имя	Владелец	Расположение
idxspace	postgres3	/var/db/postgres4/mgg73
pg_default	postgres4	
pg_global	postgres4	

(3 строки)

Владелец пользовательского табличного пространства `idxspace` после восстановления остался `postgres3`, поскольку это значение было сохранено в SQL Dump-архиве. При этом физическое расположение табличного пространства было скорректировано на путь резервного узла `/var/db/postgres4/mgg73`.

Далее была проверена доступность восстановленной базы данных и наличие пользовательских таблиц:

```

[postgres4@pg121 ~]$ psql -p "$PGPORT" -d fastorangecity -c '\conninfo'
Вы подключены к базе данных "fastorangecity" как пользователь "postgres4" через
сокет в "/tmp", порт "9530".

```

```
[postgres4@pg121 ~]$ psql -p "$PGPORT" -d fastorangecity -c '\dt'
```

Схема	Имя	Тип	Владелец
public	accounts	таблица	default_user
public	branches	таблица	default_user
public	clients	таблица	default_user
public	operations	таблица	default_user

(4 строки)

Количество восстановленных строк совпало с тестовыми данными из предыдущей лабораторной работы:

```
[postgres4@pg121 ~]$ psql -p "$PGPORT" -d fastorangecity -c "
SELECT 'clients' AS table_name, count(*) FROM clients
UNION ALL
SELECT 'branches', count(*) FROM branches
UNION ALL
SELECT 'accounts', count(*) FROM accounts
UNION ALL
SELECT 'operations', count(*) FROM operations;
"
```

table_name	count
clients	3
branches	3
accounts	3
operations	4

(4 строки)

Отдельно была проверена корректность размещения таблиц и индексов по табличным пространствам:

```
[postgres4@pg121 ~]$ psql -p "$PGPORT" -d fastorangecity -c "
SELECT
  c.relname AS object_name,
  CASE c.relkind
    WHEN 'r' THEN 'table'
    WHEN 'i' THEN 'index'
    WHEN 'S' THEN 'sequence'
    ELSE c.relkind::text
  END AS object_type,
  COALESCE(t.spcname, 'pg_default') AS tablespace
FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
LEFT JOIN pg_tablespace t ON t.oid = c.reltablespace
WHERE n.nspname = 'public'
  AND c.relname IN (
    'clients', 'branches', 'accounts', 'operations',
    'idx_accounts_client_id', 'idx_accounts_branch_id',
    'idx_operations_account_id', 'idx_operations_operation_time'
  )
ORDER BY object_type, object_name;
"
```

object_name	object_type	tablespace
idx_accounts_branch_id	index	idxspace
idx_accounts_client_id	index	idxspace
idx_operations_account_id	index	idxspace
idx_operations_operation_time	index	idxspace

accounts	table	pg_default
branches	table	pg_default
clients	table	pg_default
operations	table	pg_default

(8 строк)

Для финальной проверки было выполнено подключение к восстановленной базе от имени пользовательской роли `default_user`, то есть не от имени администратора восстановленного кластера:

```
[postgres4@pg121 ~]$ psql -h localhost -p "$PGPORT" -U default_user -d
fastorangecity -c '\conninfo'
Вы подключены к базе данных "fastorangecity" как пользователь "default_user"
(сервер "localhost": адрес "::1", порт "9530").
```

После подключения от имени `default_user` были прочитаны данные из таблиц `clients` и `operations`:

```
[postgres4@pg121 ~]$ psql -h localhost -p "$PGPORT" -U default_user -d
fastorangecity -c "SELECT * FROM clients;"
 client_id | surname | name  | gender | registered_at
-----+-----+-----+-----+-----
          1 | Иванов | Иван  | MALE   | 2026-03-01 10:00:00
          2 | Петрова | Анна  | FEMALE | 2026-03-02 11:30:00
          3 | Сидоров | Максим | NOT STATED | 2026-03-03 09:15:00
(3 строки)

[postgres4@pg121 ~]$ psql -h localhost -p "$PGPORT" -U default_user -d
fastorangecity -c "SELECT * FROM operations;"
 operation_id | account_id | operation_type | amount | operation_time
-----+-----+-----+-----+-----
            1 |          1 | Cash deposit   | 5000.00 | 2026-03-08 10:00:00
            2 |          1 | Cash withdrawal | 1000.00 | 2026-03-09 13:00:00
            3 |          2 | Credit         | 7000.00 | 2026-03-10 15:30:00
            4 |          3 | Contribution   | 2500.00 | 2026-03-11 09:40:00
(4 строки)
```

2.6. Анализ результатов этапа

В результате выполнения второго этапа была смоделирована ситуация полной недоступности основного узла `pg111`. Восстановление выполнялось исключительно на резервном узле `pg121` с использованием SQL Dump-архива, созданного на первом этапе.

На резервном узле был инициализирован новый кластер PostgreSQL с локалью `ru_RU.CP1251` и кодировкой `WIN1251`. Конфигурация порта, локальных подключений, параметров памяти и журналирования была выполнена аналогично основному узлу из предыдущей лабораторной работы. После запуска сервера резервный экземпляр стал доступен через Unix-domain socket и TCP/IP localhost.

При восстановлении была выполнена замена пути пользовательского табличного пространства `idxspace`, так как исходный путь `/var/db/postgres3/mgg73` относится к основному узлу и недоступен на резервном. После замены объекты, которые в исходном кластере размещались в `idxspace`, были восстановлены в каталоге `/var/db/postgres4/mgg73`.

Проверка показала, что база `fastorangecity` восстановлена, её владельцем является `default_user`, пользовательские таблицы `clients`, `branches`, `accounts` и `operations` доступны, а

количество строк соответствует данным из предыдущей лабораторной работы. Вторичные индексы были восстановлены в табличном пространстве `idxspace`, а сами таблицы остались в `pg_default`.

Дополнительная проверка подключения от имени `default_user` подтвердила, что восстановленная база доступна не только администратору кластера, но и пользовательской роли, от имени которой в предыдущей лабораторной работе выполнялось наполнение базы. Таким образом, при полной недоступности основного узла работа СУБД была успешно восстановлена на резервном узле.

Этап 3. Повреждение файлов БД

На данном этапе был рассмотрен сценарий повреждения файлов базы данных при сохранении доступности основного узла `pg111`. В качестве повреждаемого объекта было выбрано пользовательское табличное пространство `idxspace`, созданное в предыдущей лабораторной работе для размещения вторичных индексов.

Согласно условию задания, после повреждения необходимо проверить работу СУБД, проанализировать результат, а затем выполнить полное восстановление данных из резервной копии на основном узле. При этом исходное расположение `PGDATA` считается недоступным, поэтому восстановление выполнялось в новый каталог данных.

3.1. Проверка исходного состояния основного узла

На основном узле были заданы переменные окружения для исходного экземпляра PostgreSQL:

```
[postgres3@pg111 ~]$ export PGDATA="$HOME/ymy46"
[postgres3@pg111 ~]$ export PGPORT="9530"
[postgres3@pg111 ~]$ export PGDATABASE="fastorangecity"
```

Перед началом проверки сервер PostgreSQL был запущен:

```
[postgres3@pg111 ~]$ pg_ctl -D "$PGDATA" status
pg_ctl: сервер не работает
[postgres3@pg111 ~]$ pg_ctl -D "$PGDATA" start
ожидание запуска сервера....2026-05-01 13:26:45.807 MSK [42882] СООБЩЕНИЕ:  передача
вывода в протокол процессу сбора протоколов
готово
сервер запущен
```

Доступность базы `fastorangecity` была проверена с помощью `psql`:

```
[postgres3@pg111 ~]$ psql -p "$PGPORT" -d fastorangecity -c '\conninfo'
Вы подключены к базе данных "fastorangecity" как пользователь "postgres3" через сокет
в "/tmp", порт "9530".
```

Далее были проверены пользовательские таблицы и количество строк в них:

```
[postgres3@pg111 ~]$ psql -p "$PGPORT" -d fastorangecity -c '\dt'
```

Список отношений

Схема	Имя	Тип	Владелец
public	accounts	таблица	default_user
public	branches	таблица	default_user
public	clients	таблица	default_user
public	operations	таблица	default_user

(4 строки)

table_name | count

clients	3
branches	3
accounts	3
operations	4

(4 строки)

Также был проверен список табличных пространств. Пользовательское табличное пространство `idxspace` располагалось в каталоге `/var/db/postgres3/mgg73`:

```
[postgres3@pg111 ~]$ psql -p "$PGPORT" -d postgres -c '\db'
```

Список табличных пространств		
Имя	Владелец	Расположение
idxspace	postgres3	/var/db/postgres3/mgg73
pg_default	postgres3	
pg_global	postgres3	
(3 строки)		

Проверка объектов показала, что вторичные индексы таблиц `accounts` и `operations` размещены в `idxspace`, а таблицы находятся в `pg_default`:

object_name	object_type	tablespace
idx_accounts_branch_id	index	idxspace
idx_accounts_client_id	index	idxspace
idx_operations_account_id	index	idxspace
idx_operations_operation_time	index	idxspace
accounts	table	pg_default
branches	table	pg_default

clients	table	pg_default
operations	table	pg_default

На уровне файловой системы директория табличного пространства существовала и содержала файлы объектов:

```
[postgres3@pg111 ~]$ ls -ld "$HOME/mgg73"
drwx----- 3 postgres3 postgres 3 13 апр. 19:57 /var/db/postgres3/mgg73
[postgres3@pg111 ~]$ find "$HOME/mgg73" -maxdepth 3 -type f | head -n 20
/var/db/postgres3/mgg73/PG_16_202307071/16385/16443
/var/db/postgres3/mgg73/PG_16_202307071/16385/16440
/var/db/postgres3/mgg73/PG_16_202307071/16385/16442
/var/db/postgres3/mgg73/PG_16_202307071/16385/16441
```

3.2. Симуляция сбоя

Для симуляции повреждения файлов БД была удалена директория пользовательского табличного пространства idxspace со всем содержимым:

```
[postgres3@pg111 ~]$ rm -rf "$HOME/mgg73"
[postgres3@pg111 ~]$ ls -ld "$HOME/mgg73"
ls: /var/db/postgres3/mgg73: No such file or directory
```

После удаления директории часть данных оставалась доступной. Например, таблица clients была успешно прочитана, так как сама таблица размещалась в стандартном табличном пространстве pg_default:

```
[postgres3@pg111 ~]$ psql -p "$PGPORT" -d fastorangecity -c "SELECT * FROM clients;"
 client_id | surname | name  | gender | registered_at
-----+-----+-----+-----+-----
          1 | Иванов | Иван  | MALE   | 2026-03-01 10:00:00
          2 | Петрова | Анна  | FEMALE | 2026-03-02 11:30:00
          3 | Сидоров | Максим | NOT STATED | 2026-03-03 09:15:00
(3 строки)
```

Однако при обращении к объектам, связанным с удалённым табличным пространством, возникли ошибки отсутствия физических файлов:

```
[postgres3@pg111 ~]$ psql -p "$PGPORT" -d fastorangecity -c "SELECT * FROM operations;"
ОШИБКА: не удалось открыть файл "pg_tblspc/16386/PG_16_202307071/16385/16442": No such file or directory

[postgres3@pg111 ~]$ psql -p "$PGPORT" -d fastorangecity -c "SET enable_seqscan = off; EXPLAIN ANALYZE SELECT * FROM accounts WHERE client_id = 1;"
SET
```

```
ОШИБКА: не удалось открыть файл "pg_tblspc/16386/PG_16_202307071/16385/16440": No such file or directory
```

После перезапуска повреждённый экземпляр PostgreSQL продолжил запускаться, однако проблема с файлами табличного пространства сохранилась:

```
[postgres3@pg111 ~]$ pg_ctl -D "$PGDATA" restart
ожидание завершения работы сервера.... готово
сервер остановлен

ожидание запуска сервера....2026-05-01 13:28:42.225 MSK [43038] СООБЩЕНИЕ: передача
вывода в протокол процессу сбора протоколов

готово

сервер запущен

[postgres3@pg111 ~]$ psql -p "$PGPORT" -d fastorangecity -c "SET enable_seqscan =
off; EXPLAIN ANALYZE SELECT * FROM accounts WHERE client_id = 1;"

SET

ОШИБКА: не удалось открыть файл "pg_tblspc/16386/PG_16_202307071/16385/16440": No
such file or directory
```

В журнале сервера также появились сообщения о невозможности открыть каталог табличного пространства:

```
2026-05-01 13:28:42.246 MSK [43038] СООБЩЕНИЕ: не удалось открыть каталог
"pg_tblspc/16386/PG_16_202307071": No such file or directory

2026-05-01 13:28:50.398 MSK [43056] ОШИБКА: не удалось открыть файл
"pg_tblspc/16386/PG_16_202307071/16385/16440": No such file or directory
```

3.3. Подготовка восстановления в новый PGDATA

Согласно условию задания, исходное расположение PGDATA считается недоступным. Поэтому повреждённый экземпляр был остановлен, а восстановление выполнялось в новый каталог данных /var/db/postgres3/ymy46_stage3_restored. Также были подготовлены новые каталоги для WAL и табличного пространства idxspace.

```
[postgres3@pg111 ~]$ export OLD_PGDATA="$HOME/ymy46"
[postgres3@pg111 ~]$ export PGDATA="$HOME/ymy46_stage3_restored"
[postgres3@pg111 ~]$ export PGWAL="$HOME/xlg69_stage3_restored"
[postgres3@pg111 ~]$ export NEW_TABLESPACE_DIR="$HOME/mgg73_stage3_restored"
[postgres3@pg111 ~]$ export PGPORT="9530"
[postgres3@pg111 ~]$ export PGDATABASE="fastorangecity"
[postgres3@pg111 ~]$ export BACKUP_DIR_REMOTE="/var/db/postgres4/pg_backups/sql_dump"
```

Повреждённый старый кластер был остановлен:

```
[postgres3@pg111 ~]$ pg_ctl -D "$OLD_PGDATA" stop -m fast
ожидание завершения работы сервера.... готово
```

сервер остановлен

```
[postgres3@pg111 ~]$ pg_ctl -D "$OLD_PGDATA" status
```

pg_ctl: сервер не работает

Затем были подготовлены новые каталоги восстановления:

```
[postgres3@pg111 ~]$ rm -rf "$PGDATA" "$PGWAL" "$NEW_TABLESPACE_DIR"
```

```
[postgres3@pg111 ~]$ mkdir -p "$PGWAL" "$NEW_TABLESPACE_DIR"
```

```
[postgres3@pg111 ~]$ chmod 700 "$PGWAL" "$NEW_TABLESPACE_DIR"
```

```
[postgres3@pg111 ~]$ ls -ld "$PGWAL" "$NEW_TABLESPACE_DIR"
```

```
drwx----- 2 postgres3 postgres 2  1 мая  13:30  
/var/db/postgres3/mgg73_stage3_restored
```

```
drwx----- 2 postgres3 postgres 2  1 мая  13:30  
/var/db/postgres3/xlg69_stage3_restored
```

Для восстановления был выбран последний SQL Dump-архив, находящийся на резервном узле pg121. Целостность архива была проверена перед восстановлением:

```
[postgres3@pg111 ~]$ LATEST_BACKUP="$(ssh backup_pg121 "ls -t  
$BACKUP_DIR_REMOTE/pg111_full_cluster_*.sql.gz | head -n 1")"
```

```
[postgres3@pg111 ~]$ echo "$LATEST_BACKUP"
```

```
/var/db/postgres4/pg_backups/sql_dump/pg111_full_cluster_2026-04-27_03-00-00.sql.gz
```

```
[postgres3@pg111 ~]$ ssh backup_pg121 "gzip -t '$LATEST_BACKUP' && echo OK"
```

OK

3.4. Инициализация и конфигурация нового кластера

Новый кластер PostgreSQL был инициализирован с той же локалью и кодировкой, что и исходный кластер:

```
[postgres3@pg111 ~]$ initdb -D "$PGDATA" -X "$PGWAL" --locale=ru_RU.CP1251 --  
encoding=WIN1251
```

Файлы, относящиеся к этой СУБД, будут принадлежать пользователю "postgres3".

Кластер баз данных будет инициализирован с локалью "ru_RU.CP1251".

Выбрана конфигурация текстового поиска по умолчанию "russian".

создание каталога /var/db/postgres3/ymy46_stage3_restored... ок

исправление прав для существующего каталога
/var/db/postgres3/xlg69_stage3_restored... ок

создание конфигурационных файлов... ок

сохранение данных на диске... ок

Готово. Теперь вы можете запустить сервер баз данных:

```
pg_ctl -D /var/db/postgres3/ymy46_stage3_restored -l файл_журнала start
```

После инициализации в postgresql.conf были добавлены параметры, аналогичные исходному кластеру:

```
[postgres3@pg111 ~]$ cat >> "$PGDATA/postgresql.conf" <<'EOF'
# Настройки восстановленного экземпляра для этапа 3
listen_addresses = 'localhost'
port = 9530
max_connections = 30
shared_buffers = 2GB
temp_buffers = 32MB
work_mem = 64MB
checkpoint_timeout = 15min
effective_cache_size = 18GB
fsync = on
commit_delay = 10000
logging_collector = on
log_destination = 'stderr'
log_directory = 'log'
log_filename = 'postgresql-%Y-%m-%d_%H%M%S.log'
log_min_messages = notice
log_connections = on
log_checkpoints = on
EOF
```

Файл pg_hba.conf был настроен для локальных подключений через Unix-domain socket и TCP/IP localhost:

```
[postgres3@pg111 ~]$ cat > "$PGDATA/pg_hba.conf" <<'EOF'
# TYPE DATABASE USER ADDRESS METHOD
local all all peer
host all all 127.0.0.1/32 trust
host all all ::1/128 trust
host all all 0.0.0.0/0 reject
host all all ::0/0 reject
EOF
```

Новый пустой кластер был успешно запущен:

```
[postgres3@pg111 ~]$ pg_ctl -D "$PGDATA" start
ожидание запуска сервера....2026-05-01 13:32:46.405 MSK [43605] СООБЩЕНИЕ: передача
вывода в протокол процессу сбора протоколов
готово
сервер запущен
```

```
[postgres3@pg111 ~]$ pg_ctl -D "$PGDATA" status
pg_ctl: сервер работает (PID: 43605)
/usr/local/bin/postgres "-D" "/var/db/postgres3/ymy46_stage3_restored"
[postgres3@pg111 ~]$ psql -p "$PGPORT" -d postgres -c '\conninfo'
Вы подключены к базе данных "postgres" как пользователь "postgres3" через сокет в
"/tmp", порт "9530".
```

3.5. Восстановление данных из резервной копии

При первой попытке восстановления возникла ошибка, так как роль postgres3 уже была создана автоматически при инициализации нового кластера, а SQL Dump-архив также содержал команду CREATE ROLE postgres3. Из-за параметра ON_ERROR_STOP=1 восстановление было остановлено до создания базы fastorangecity:

```
[postgres3@pg111 ~]$ ssh backup_pg121 "gzip -dc '$LATEST_BACKUP'" | sed
"s|/var/db/postgres3/mgg73|/var/db/postgres3/mgg73_stage3_restored|g" | psql -p
"$PGPORT" -d postgres -v ON_ERROR_STOP=1
SET
SET
SET
CREATE ROLE
ALTER ROLE
ОШИБКА: роль "postgres3" уже существует
```

Для корректного восстановления новый кластер был пересоздан, после чего из потока восстановления были исключены команды CREATE ROLE postgres3 и ALTER ROLE postgres3. Это безопасно, так как роль postgres3 уже существует в новом кластере. Одновременно путь табличного пространства был заменён со старого /var/db/postgres3/mgg73 на новый /var/db/postgres3/mgg73_stage3_restored.

```
[postgres3@pg111 ~]$ ssh backup_pg121 "gzip -dc '$LATEST_BACKUP'" \
| sed \
-e "s|/var/db/postgres3/mgg73|/var/db/postgres3/mgg73_stage3_restored|g" \
-e "/^CREATE ROLE postgres3;$/d" \
-e "/^ALTER ROLE postgres3 /d" \
| psql -p "$PGPORT" -d postgres -v ON_ERROR_STOP=1
SET
SET
SET
CREATE ROLE
ALTER ROLE
CREATE TABLESPACE
GRANT
```

```

CREATE DATABASE
ALTER DATABASE
CREATE TABLE
CREATE SEQUENCE
COPY 3
COPY 3
COPY 3
COPY 4
CREATE INDEX
CREATE INDEX
CREATE INDEX
CREATE INDEX

```

3.6. Проверка результата восстановления

После восстановления был проверен список баз данных. База fastorangecity снова присутствует в кластере, её владельцем является default_user:

```

[postgres3@pg111 ~]$ psql -p "$PGPORT" -d postgres -c '\l'

```

Список баз данных					
Имя	Владелец	Кодировка	Провайдер локали	LC_COLLATE	LC_CTYPE
fastorangecity	default_user	WIN1251	libc	ru_RU.CP1251	ru_RU.CP1251
postgres	postgres3	WIN1251	libc	ru_RU.CP1251	ru_RU.CP1251
template0	postgres3	WIN1251	libc	ru_RU.CP1251	ru_RU.CP1251
template1	postgres3	WIN1251	libc	ru_RU.CP1251	ru_RU.CP1251

(4 строки)

Табличное пространство idxspace было восстановлено в новую директорию, что соответствует условию о недоступности исходного расположения:

```

[postgres3@pg111 ~]$ psql -p "$PGPORT" -d postgres -c '\db'

```

Список табличных пространств		
Имя	Владелец	Расположение
idxspace	postgres3	/var/db/postgres3/mgg73_stage3_restored
pg_default	postgres3	


```
pg_global | postgres3 |
```

(3 строки)

Была проверена доступность базы fastorangecity и наличие пользовательских таблиц:

```
[postgres3@pg111 ~]$ psql -p "$PGPORT" -d fastorangecity -c '\conninfo'
```

Вы подключены к базе данных "fastorangecity" как пользователь "postgres3" через сокет в "/tmp", порт "9530".

```
[postgres3@pg111 ~]$ psql -p "$PGPORT" -d fastorangecity -c '\dt'
```

Список отношений

Схема	Имя	Тип	Владелец
public	accounts	таблица	default_user
public	branches	таблица	default_user
public	clients	таблица	default_user
public	operations	таблица	default_user

(4 строки)

Количество строк в таблицах после восстановления совпало с исходными данными:

```
table_name | count
```

table_name	count
clients	3
branches	3
accounts	3
operations	4

(4 строки)

Контрольный запрос, который до восстановления завершался ошибкой отсутствия файла индекса, после восстановления успешно выполнялся через Index Scan с использованием индекса `idx_accounts_client_id`:

```
[postgres3@pg111 ~]$ psql -p "$PGPORT" -d fastorangecity -c "SET enable_seqscan = off; EXPLAIN ANALYZE SELECT * FROM accounts WHERE client_id = 1;"
```

SET

QUERY PLAN

Index Scan using `idx_accounts_client_id` on `accounts` (cost=0.13..8.15 rows=1 width=48) (actual time=0.048..0.049 rows=1 loops=1)

Index Cond: (`client_id` = 1)

Planning Time: 0.356 ms

Execution Time: 0.091 ms

3.7. Анализ результатов этапа

После удаления директории `/var/db/postgres3/mgg73` сервер PostgreSQL продолжил запускаться, так как основной каталог данных PGDATA остался доступен. Однако объекты, физически размещённые в удалённом табличном пространстве `idxspace`, стали недоступны. Это проявилось при обращении к индексам: запросы, использующие индексы `idx_accounts_client_id` и `idx_operations_account_id`, завершались ошибкой `No such file or directory`.

После перезапуска сервер также запустился, но в журнале появились сообщения о невозможности открыть каталог `pg_tblspc/16386/PG_16_202307071`. Это означает, что PostgreSQL сохранил метаданные о табличном пространстве и объектах в системном каталоге, но соответствующие физические файлы на диске отсутствовали.

Для восстановления исходный каталог PGDATA был признан недоступным. Повреждённый экземпляр был остановлен, после чего создан новый каталог данных `/var/db/postgres3/ymy46_stage3_restored`, новый каталог WAL `/var/db/postgres3/xlg69_stage3_restored` и новый каталог для табличного пространства `/var/db/postgres3/mgg73_stage3_restored`.

Восстановление выполнялось из SQL Dump-архива, хранившегося на резервном узле `pg121`. Так как в архиве был сохранён старый путь табличного пространства `/var/db/postgres3/mgg73`, при восстановлении он был заменён на новый путь `/var/db/postgres3/mgg73_stage3_restored`. Кроме того, из потока восстановления были исключены команды `CREATE ROLE postgres3` и `ALTER ROLE postgres3`, поскольку роль `postgres3` уже была создана автоматически при инициализации нового кластера.

После восстановления база `fastorangecity`, таблицы `clients`, `branches`, `accounts` и `operations`, пользовательская роль `default_user` и табличное пространство `idxspace` были восстановлены. Количество строк в таблицах совпало с исходным состоянием. Контрольный запрос с принудительным использованием индекса `idx_accounts_client_id` успешно выполнялся через `Index Scan`, что подтвердило восстановление физических файлов индексов в новом табличном пространстве.

Этап 4. Логическое повреждение данных

На данном этапе был рассмотрен сценарий логического повреждения данных при сохранении работоспособности основного узла. В качестве логической ошибки была смоделирована нежелательная операция DROP TABLE, удаляющая две таблицы пользовательской базы. Восстановление выполнялось на основном узле с использованием архивных WAL-файлов и восстановления на момент времени (PITR).

4.1. Подготовка основного узла и включение архивирования WAL

В качестве основного экземпляра для этапа 4 использовался восстановленный после третьего этапа кластер PostgreSQL. Были заданы переменные окружения для каталога данных, каталога WAL, пользовательского табличного пространства, каталога архива WAL и каталогов будущей базовой копии:

```
[postgres3@pg111 ~]$ export PGDATA="$HOME/ymy46_stage3_restored"
[postgres3@pg111 ~]$ export PGWAL="$HOME/xlg69_stage3_restored"
[postgres3@pg111 ~]$ export PGPORT="9530"
[postgres3@pg111 ~]$ export PGDATABASE="fastorangecity"
[postgres3@pg111 ~]$ export TABLESPACE_DIR="$HOME/mgg73_stage3_restored"

[postgres3@pg111 ~]$ export ARCHIVE_DIR="$HOME/wal_archive_stage4"

[postgres3@pg111 ~]$ export BASE_COPY_DIR="$HOME/stage4_basecopy"
[postgres3@pg111 ~]$ export BASE_PGDATA="$BASE_COPY_DIR/ymy46_base"
[postgres3@pg111 ~]$ export BASE_PGWAL="$BASE_COPY_DIR/xlg69_base"
[postgres3@pg111 ~]$ export BASE_TABLESPACE_DIR="$BASE_COPY_DIR/mgg73_base"

[postgres3@pg111 ~]$ export RESTORE_PGDATA="$HOME/ymy46_stage4_restored"
[postgres3@pg111 ~]$ export RESTORE_PGWAL="$HOME/xlg69_stage4_restored"
[postgres3@pg111 ~]$ export RESTORE_TABLESPACE_DIR="$HOME/mgg73_stage4_restored"
```

Перед настройкой архивирования была проверена работоспособность текущего кластера:

```
[postgres3@pg111 ~]$ pg_ctl -D "$PGDATA" status
pg_ctl: сервер работает (PID: 43605)
/usr/local/bin/postgres "-D" "/var/db/postgres3/ymy46_stage3_restored"

[postgres3@pg111 ~]$ psql -p "$PGPORT" -d fastorangecity -c '\conninfo'
Вы подключены к базе данных "fastorangecity" как пользователь "postgres3" через
сокет в "/tmp", порт "9530".
```

Для хранения архивных WAL-файлов был создан отдельный каталог:

```
[postgres3@pg111 ~]$ mkdir -p "$ARCHIVE_DIR"
[postgres3@pg111 ~]$ chmod 700 "$ARCHIVE_DIR"
[postgres3@pg111 ~]$ ls -ld "$ARCHIVE_DIR"
drwx----- 2 postgres3 postgres3 2 1 мая 14:36
/var/db/postgres3/wal_archive_stage4
```

В файл postgresql.conf были добавлены параметры архивирования WAL:

```
[postgres3@pg111 ~]$ cat >> "$PGDATA/postgresql.conf" <<'EOF'

# Настройки архивирования WAL для этапа 4
```

```
archive_mode = on
archive_command = 'test ! -f /var/db/postgres3/wal_archive_stage4/%f && cp %p
/var/db/postgres3/wal_archive_stage4/%f'
archive_timeout = 60s
EOF
```

Параметр `archive_mode` требует перезапуска сервера, поэтому кластер был перезапущен и настройки были проверены:

```
[postgres3@pg111 ~]$ pg_ctl -D "$PGDATA" restart
ожидание завершения работы сервера.... готово
сервер остановлен
ожидание запуска сервера....2026-05-01 14:36:39.338 MSK [50294] СООБЩЕНИЕ:
передача вывода в протокол процессу сбора протоколов
готово
сервер запущен

[postgres3@pg111 ~]$ psql -p "$PGPORT" -d postgres -c "SHOW archive_mode;"
archive_mode
-----
on
(1 строка)

[postgres3@pg111 ~]$ psql -p "$PGPORT" -d postgres -c "SHOW archive_command;"
archive_command
-----
test ! -f /var/db/postgres3/wal_archive_stage4/%f && cp %p
/var/db/postgres3/wal_archive_stage4/%f
(1 строка)

[postgres3@pg111 ~]$ psql -p "$PGPORT" -d postgres -c "SHOW archive_timeout;"
archive_timeout
-----
1min
(1 строка)
```

Работа архивирования была проверена принудительным переключением WAL. После выполнения `pg_switch_wal()` в каталоге архива появился WAL-файл размером 16 МБ:

```
[postgres3@pg111 ~]$ psql -p "$PGPORT" -d postgres -c "SELECT pg_switch_wal();"
pg_switch_wal
-----
0/19B4910
(1 строка)

[postgres3@pg111 ~]$ ls -lh "$ARCHIVE_DIR" | tail
total 1
-rw----- 1 postgres3 postgres 16M 1 мая 14:36 00000001000000000000000001
```

4.2. Создание физической базовой копии

Для восстановления на момент времени одной только последовательности WAL недостаточно: нужна базовая физическая копия, от которой будут применяться архивные WAL-файлы. Перед копированием был выполнен CHECKPOINT, затем сервер был остановлен.

```
[postgres3@pg111 ~]$ psql -p "$PGPORT" -d postgres -c "CHECKPOINT;"
CHECKPOINT

[postgres3@pg111 ~]$ pg_ctl -D "$PGDATA" stop -m fast
ожидание завершения работы сервера.... готово
сервер остановлен
```

После остановки сервера были скопированы каталог данных, отдельный каталог WAL и каталог пользовательского табличного пространства:

```
[postgres3@pg111 ~]$ rm -rf "$BASE_COPY_DIR"
[postgres3@pg111 ~]$ mkdir -p "$BASE_COPY_DIR"

[postgres3@pg111 ~]$ cp -a "$PGDATA" "$BASE_PGDATA"
[postgres3@pg111 ~]$ cp -a "$PGWAL" "$BASE_PGWAL"
[postgres3@pg111 ~]$ cp -a "$TABLESPACE_DIR" "$BASE_TABLESPACE_DIR"

[postgres3@pg111 ~]$ ls -ld "$BASE_PGDATA" "$BASE_PGWAL" "$BASE_TABLESPACE_DIR"
drwx----- 3 postgres3 postgres 3 1 мая 13:32
/var/db/postgres3/stage4_basecopy/mgg73_base
drwx----- 3 postgres3 postgres 5 1 мая 14:37
/var/db/postgres3/stage4_basecopy/xlg69_base
drwx----- 19 postgres3 postgres 27 1 мая 14:37
/var/db/postgres3/stage4_basecopy/ymy46_base
```

После создания базовой копии основной кластер был запущен снова:

```
[postgres3@pg111 ~]$ pg_ctl -D "$PGDATA" start
ожидание запуска сервера....2026-05-01 14:37:42.460 MSK [50356] СООБЩЕНИЕ:
передача вывода в протокол процессу сбора протоколов
готово
сервер запущен

[postgres3@pg111 ~]$ pg_ctl -D "$PGDATA" status
pg_ctl: сервер работает (PID: 50356)
/usr/local/bin/postgres "-D" "/var/db/postgres3/ymy46_stage3_restored"

[postgres3@pg111 ~]$ psql -p "$PGPORT" -d fastorangecity -c '\conninfo'
Вы подключены к базе данных "fastorangecity" как пользователь "postgres3" через
сокет в "/tmp", порт "9530".
```

4.3. Добавление новых строк и фиксация точки восстановления

После создания базовой копии в каждую пользовательскую таблицу были добавлены новые строки. Эти изменения должны сохраниться после восстановления, так как целевая точка восстановления будет выбрана после вставки данных, но до ошибочного удаления таблиц.

```
[postgres3@pg111 ~]$ psql -h localhost -p "$PGPORT" -U default_user -d
fastorangecity <<'SQL'
INSERT INTO clients (surname, name, gender, registered_at) VALUES
('Кузнецов', 'Алексей', 'MALE', '2026-05-01 13:40:00'),
('Смирнова', 'Елена', 'FEMALE', '2026-05-01 13:41:00');

INSERT INTO branches (branch_name, city, opened_at) VALUES
('Западный филиал', 'Калининград', '2023-02-10'),
('Восточный филиал', 'Владивосток', '2023-03-15');

INSERT INTO accounts (client_id, branch_id, balance, opened_at) VALUES
((SELECT client_id FROM clients WHERE surname = 'Кузнецов'),
 (SELECT branch_id FROM branches WHERE branch_name = 'Западный филиал'),
 31000.00, '2026-05-01 13:42:00'),
((SELECT client_id FROM clients WHERE surname = 'Смирнова'),
 (SELECT branch_id FROM branches WHERE branch_name = 'Восточный филиал'),
 42000.00, '2026-05-01 13:43:00');

INSERT INTO operations (account_id, operation_type, amount, operation_time) VALUES
((SELECT a.account_id FROM accounts a JOIN clients c ON c.client_id = a.client_id
WHERE c.surname = 'Кузнецов'),
 'Cash deposit', 6000.00, '2026-05-01 13:44:00'),
((SELECT a.account_id FROM accounts a JOIN clients c ON c.client_id = a.client_id
WHERE c.surname = 'Смирнова'),
 'Contribution', 8000.00, '2026-05-01 13:45:00');
SQL
INSERT 0 2
INSERT 0 2
INSERT 0 2
INSERT 0 2
```

После вставки данных было проверено количество строк в таблицах:

```
[postgres3@pg111 ~]$ psql -p "$PGPORT" -d fastorangecity -c "
SELECT 'clients' AS table_name, count(*) FROM clients
UNION ALL
SELECT 'branches', count(*) FROM branches
UNION ALL
SELECT 'accounts', count(*) FROM accounts
UNION ALL
SELECT 'operations', count(*) FROM operations;
"
 table_name | count
-----+-----
 clients   |      5
 branches  |      5
 accounts  |      5
 operations |      6
(4 строки)
```

Далее была зафиксирована точка восстановления. Она находится после вставки новых строк и до выполнения ошибочной операции DROP TABLE:

```
[postgres3@pg111 ~]$ psql -p "$PGPORT" -d postgres -c "SELECT clock_timestamp() AS
recovery_target_time;"
 recovery_target_time
-----
```

```
2026-05-01 14:37:58.468081+03
(1 строка)
```

4.4. Симуляция логической ошибки

Для симуляции логического повреждения данных были удалены две таблицы: operations и accounts.

```
[postgres3@pg111 ~]$ psql -h localhost -p "$PGPORT" -U default_user -d
fastorangecity -c "
DROP TABLE operations;
DROP TABLE accounts;
"
DROP TABLE
DROP TABLE
```

После ошибочной операции в базе остались только таблицы branches и clients:

```
[postgres3@pg111 ~]$ psql -p "$PGPORT" -d fastorangecity -c '\dt'
          Список отношений
 Схема |   Имя   | Тип   | Владелец
-----+-----+-----+-----
public | branches | таблица | default_user
public | clients  | таблица | default_user
(2 строки)
```

Попытка обратиться к удалённой таблице accounts завершилась ошибкой:

```
[postgres3@pg111 ~]$ psql -p "$PGPORT" -d fastorangecity -c "SELECT * FROM
accounts;"
ОШИБКА: отношение "accounts" не существует
СТРОКА 1: SELECT * FROM accounts;
      ^
```

Чтобы WAL с ошибочной операцией был гарантированно архивирован, было выполнено переключение WAL. После этого в каталоге архива присутствовали несколько WAL-файлов:

```
[postgres3@pg111 ~]$ psql -p "$PGPORT" -d postgres -c "SELECT pg_switch_wal();"
pg_switch_wal
-----
0/301D258
(1 строка)

[postgres3@pg111 ~]$ ls -lh "$ARCHIVE_DIR" | tail
total 3898
-rw----- 1 postgres3 postgres 16M 1 мая 14:36 00000001000000000000000001
-rw----- 1 postgres3 postgres 16M 1 мая 14:37 00000001000000000000000002
-rw----- 1 postgres3 postgres 16M 1 мая 14:39 00000001000000000000000003
```

После демонстрации логической ошибки повреждённый кластер был остановлен:

```
[postgres3@pg111 ~]$ pg_ctl -D "$PGDATA" stop -m fast
ожидание завершения работы сервера.... готово
сервер остановлен
```

```
[postgres3@pg111 ~]$ pg_ctl -D "$PGDATA" status
pg_ctl: сервер не работает
```

4.5. Восстановление на момент времени с использованием архивных WAL

Для восстановления была использована созданная ранее базовая физическая копия. Она была развернута в новые каталоги, чтобы не использовать повреждённый рабочий экземпляр.

```
[postgres3@pg111 ~]$ rm -rf "$RESTORE_PGDATA" "$RESTORE_PGWAL"
"$RESTORE_TABLESPACE_DIR"

[postgres3@pg111 ~]$ cp -a "$BASE_PGDATA" "$RESTORE_PGDATA"
[postgres3@pg111 ~]$ cp -a "$BASE_PGWAL" "$RESTORE_PGWAL"
[postgres3@pg111 ~]$ cp -a "$BASE_TABLESPACE_DIR" "$RESTORE_TABLESPACE_DIR"

[postgres3@pg111 ~]$ ls -ld "$RESTORE_PGDATA" "$RESTORE_PGWAL"
"$RESTORE_TABLESPACE_DIR"
drwx-----  3 postgres3 postgres  3  1 мая  13:32
/var/db/postgres3/mgg73_stage4_restored
drwx-----  3 postgres3 postgres  5  1 мая  14:37
/var/db/postgres3/xlg69_stage4_restored
drwx----- 19 postgres3 postgres 27  1 мая  14:37
/var/db/postgres3/ymy46_stage4_restored
```

Так как базовая копия содержала символические ссылки на старые каталоги WAL и табличного пространства, эти ссылки были скорректированы на новые пути восстановления:

```
[postgres3@pg111 ~]$ ls -l "$RESTORE_PGDATA/pg_wal"
lrwx-----  1 postgres3 postgres 39  1 мая  13:32
/var/db/postgres3/ymy46_stage4_restored/pg_wal ->
/var/db/postgres3/xlg69_stage3_restored

[postgres3@pg111 ~]$ ls -l "$RESTORE_PGDATA/pg_tblspc"
total 1
lrwx-----  1 postgres3 postgres 39  1 мая  13:32 16385 ->
/var/db/postgres3/mgg73_stage3_restored

[postgres3@pg111 ~]$ rm -rf "$RESTORE_PGDATA/pg_wal"
[postgres3@pg111 ~]$ ln -s "$RESTORE_PGWAL" "$RESTORE_PGDATA/pg_wal"

[postgres3@pg111 ~]$ rm -f "$RESTORE_PGDATA/pg_tblspc/16385"
[postgres3@pg111 ~]$ ln -s "$RESTORE_TABLESPACE_DIR"
"$RESTORE_PGDATA/pg_tblspc/16385"

[postgres3@pg111 ~]$ ls -l "$RESTORE_PGDATA/pg_wal"
lrwxr-xr-x  1 postgres3 postgres 39  1 мая  14:39
/var/db/postgres3/ymy46_stage4_restored/pg_wal ->
/var/db/postgres3/xlg69_stage4_restored

[postgres3@pg111 ~]$ ls -l "$RESTORE_PGDATA/pg_tblspc"
total 1
```



```
lrwxr-xr-x 1 postgres3 postgres 39 1 мая 14:40 16385 ->
/var/db/postgres3/mgg73_stage4_restored
```

Для запуска восстановления был создан файл `recovery.signal`, а в `postgresql.conf` были добавлены параметры `restore_command`, `recovery_target_time` и `recovery_target_action`. В качестве `recovery_target_time` использовалось время, зафиксированное после вставки новых строк, но до удаления таблиц.

```
[postgres3@pg111 ~]$ touch "$RESTORE_PGDATA/recovery.signal"

[postgres3@pg111 ~]$ cat >> "$RESTORE_PGDATA/postgresql.conf" <<'EOF'

# Настройки восстановления PITR для этапа 4
restore_command = 'cp /var/db/postgres3/wal_archive_stage4/%f %p'
recovery_target_time = '2026-05-01 14:37:58.468081+03'
recovery_target_action = 'promote'
EOF
```

После этого восстановленный кластер был запущен:

```
[postgres3@pg111 ~]$ pg_ctl -D "$RESTORE_PGDATA" start
ожидание запуска сервера....2026-05-01 14:41:04.513 MSK [50730] СООБЩЕНИЕ:
передача вывода в протокол процессу сбора протоколов
готово
сервер запущен

[postgres3@pg111 ~]$ pg_ctl -D "$RESTORE_PGDATA" status
pg_ctl: сервер работает (PID: 50730)
/usr/local/bin/postgres "-D" "/var/db/postgres3/ymy46_stage4_restored"
```

4.6. Проверка результата восстановления

После запуска восстановленного кластера база `fastorangecity` была доступна для подключения:

```
[postgres3@pg111 ~]$ psql -p "$PGPORT" -d fastorangecity -c '\conninfo'
Вы подключены к базе данных "fastorangecity" как пользователь "postgres3" через
сокет в "/tmp", порт "9530".
```

Проверка списка таблиц показала, что удалённые таблицы `accounts` и `operations` были восстановлены:

```
[postgres3@pg111 ~]$ psql -p "$PGPORT" -d fastorangecity -c '\dt'
          Список отношений
 Схема |   Имя   | Тип   | Владелец
-----+-----+-----+-----
public | accounts | таблица | default_user
public | branches | таблица | default_user
public | clients  | таблица | default_user
public | operations | таблица | default_user
(4 строки)
```

Количество строк после восстановления соответствует состоянию после вставки новых данных и до ошибочного удаления таблиц:

```
[postgres3@pg111 ~]$ psql -p "$PGPORT" -d fastorangecity -c "
SELECT 'clients' AS table_name, count(*) FROM clients
UNION ALL
SELECT 'branches', count(*) FROM branches
UNION ALL
SELECT 'accounts', count(*) FROM accounts
UNION ALL
SELECT 'operations', count(*) FROM operations;
"
table_name | count
-----+-----
clients    |      5
branches    |      5
accounts    |      5
operations  |      6
(4 строки)
```

Дополнительно были выведены последние строки из таблиц clients и operations. Новые записи с идентификаторами 4, 5 и операции с идентификаторами 5, 6 присутствуют после восстановления:

```
[postgres3@pg111 ~]$ psql -p "$PGPORT" -d fastorangecity -c "
SELECT *
FROM clients
WHERE client_id IN (4, 5);
"
client_id | surname | name | gender | registered_at
-----+-----+-----+-----+-----
4 | РѢСІР·PSPμC†PsPI | РђР»РμРєСІРμPN° | MALE | 2026-05-01 13:40:00
5 | РЎРјРёСђPSPsPIP° | Р•Р»РμPSP° | FEMALE | 2026-05-01 13:41:00
(2 строки)

[postgres3@pg111 ~]$ psql -p "$PGPORT" -d fastorangecity -c "
SELECT *
FROM operations
WHERE operation_id IN (5, 6);
"
operation_id | account_id | operation_type | amount | operation_time
-----+-----+-----+-----+-----
5 | 4 | Cash deposit | 6000.00 | 2026-05-01 13:44:00
6 | 5 | Contribution | 8000.00 | 2026-05-01 13:45:00
(2 строки)
```

Некорректное отображение кириллических фамилий связано с несовпадением кодировки клиентского терминала и кодировки базы WIN1251. При этом факт восстановления записей подтверждается идентификаторами, количеством строк и связью операций с восстановленными счетами.

В журнале PostgreSQL зафиксировано восстановление до выбранной точки времени и завершение recovery с последующим переходом сервера в обычный режим работы:

```
2026-05-01 14:41:04.544 MSK [50734] СООБЩЕНИЕ: начинается восстановление точки во
времени до 2026-05-01 14:37:58.468081+03
2026-05-01 14:41:04.546 MSK [50734] СООБЩЕНИЕ: файл журнала
"000000010000000000000003" восстановлен из архива
```

```
2026-05-01 14:41:04.596 MSK [50734] СООБЩЕНИЕ: согласованное состояние
восстановления достигнуто в позиции 0/30000A0
2026-05-01 14:41:04.645 MSK [50734] СООБЩЕНИЕ: восстановление останавливается
перед фиксированием транзакции 795, время 2026-05-01 14:38:55.349534+03
2026-05-01 14:41:04.645 MSK [50734] СООБЩЕНИЕ: последняя завершённая транзакция
была выполнена в 2026-05-01 14:37:50.251972+03
2026-05-01 14:41:04.914 MSK [50734] СООБЩЕНИЕ: восстановление архива завершено
2026-05-01 14:41:05.284 MSK [50730] СООБЩЕНИЕ: система БД готова принимать
подключения
```

4.7. Анализ результатов этапа

В результате выполнения этапа было настроено архивирование WAL на основном узле. После включения `archive_mode` и настройки `archive_command` был выполнен `pg_switch_wal()`, после чего в каталоге `/var/db/postgres3/wal_archive_stage4` появился архивный WAL-файл. Это подтвердило работоспособность механизма архивирования.

Перед симуляцией логической ошибки была создана физическая базовая копия кластера: каталога PGDATA, каталога WAL и каталога пользовательского табличного пространства `idxspace`. После этого основной кластер был снова запущен, и в каждую пользовательскую таблицу были добавлены новые строки. Количество строк увеличилось до 5 в таблицах `clients`, `branches` и `accounts` и до 6 в таблице `operations`.

После вставки данных было зафиксировано время восстановления: 2026-05-01 14:37:58.468081+03. Затем была смоделирована логическая ошибка: из базы `fastorangecity` были удалены таблицы `operations` и `accounts`. Проверка списка отношений показала, что после ошибки в базе остались только таблицы `branches` и `clients`, а обращение к `accounts` завершалось ошибкой “отношение `accounts` не существует”.

Для восстановления была использована базовая физическая копия и архивные WAL-файлы. Восстановленный экземпляр был размещён в новом каталоге `/var/db/postgres3/my46_stage4_restored`, WAL был вынесен в `/var/db/postgres3/xlg69_stage4_restored`, а табличное пространство `idxspace` было перенаправлено на `/var/db/postgres3/mgg73_stage4_restored`. В файле `postgresql.conf` были заданы параметры `restore_command`, `recovery_target_time` и `recovery_target_action = 'promote'`, после чего сервер был запущен.

Журнал PostgreSQL подтвердил, что сервер начал восстановление до указанной точки времени, восстановил необходимые WAL-файлы из архива и остановил восстановление перед транзакцией, выполненной в 2026-05-01 14:38:55.349534+03. Последняя завершённая транзакция перед целевой точкой была выполнена в 2026-05-01 14:37:50.251972+03, то есть уже после вставки новых строк, но до ошибочного удаления таблиц. После завершения восстановления сервер был автоматически переведён в рабочий режим.

Проверка после восстановления показала, что таблицы `accounts` и `operations` снова присутствуют в базе. Количество строк составило 5 для `clients`, `branches` и `accounts` и 6 для `operations`. Это подтверждает, что новые строки, добавленные до зафиксированной точки восстановления, были сохранены, а ошибочная операция `DROP TABLE`, выполненная после неё, не была применена.